

Freight Rate Prediction

Validation, data split, and December 2025 chart

This report documents how the model was validated, how the labeled data was split, and the December chart produced by score.py. The model and preprocessing are in main.ipynb. Spotter scores the unlabeled November-December loads after submission. The numbers in section 5 are from a September-October holdout the model did not train on.

1. What the data showed

train-test.csv has 48,000 labeled loads from 1 January through 31 October 2025 (304 consecutive days, about 158 loads a day). The target is posted_rate. validation.csv is the following two months, 12,000 loads. The December file is one fixed lane for each day of the month: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb.

- Distance drives the rate. Correlation with posted_rate is 0.91. Great-circle distance from pickup and delivery coordinates matches that almost exactly.
- Weight, market_index, and quote_signal have correlations near zero on their own. Extra signal has to come from nonlinear effects, lane, and equipment.
- 64 pickup cities, 64 delivery cities, three equipment types: Dry Van 57%, Reefer 25%, Flatbed 18%.
- posted_rate is right-skewed (skew 1.90). log1p brings skew to -0.49, so the model trains on log1p(posted_rate) and predictions are inverted with expm1.

2. Data-quality issues

Issue	Finding	How it was handled
Weight	300 missing values and 292 negatives. The negatives sit in the same range as real weights (about 5,000 to 47,500 lb).	Treated as sign-flip errors and corrected with abs(). Missing values filled with the equipment median. A weight_was_missing flag is kept as a feature.
Market index	374 missing values. Nearly constant within a day (mean within-day std about 0.025) and varies by month.	Filled with the same-date mean, then a global median if a date has no values. A market_index_was_missing flag is kept.
Distance vs. coordinates	22 rows have stated distance outside 0.5x to 2x the haversine distance. Several are short lanes recorded at 70 miles.	Rows were kept. haversine_distance and distance_ratio are features, so the model can see the mismatch.

Load IDs are unique and match the expected format. Latitudes and longitudes are in range, and each of the 64 cities maps to one coordinate pair. rate_per_mile (posted_rate / distance) was used only in exploration and is not a model feature. Imputation statistics used at prediction time were fit on the labeled data and reused for validation.csv and the December file.

3. Why this model

The baseline is a linear regression on distance alone, the only strong linear signal. On the holdout it reaches a mean absolute error of \$494, a MAPE of 23%, and an R-squared of 0.58.

The submitted model is XGBoost. Lane, equipment, seasonality, and the weaker numeric features are nonlinear, and trees can use pickup, delivery, equipment, and lane as native categories without target encoding.

Training uses squared error on the log rate, histogram splits, learning rate 0.03, max depth 6, row and column subsampling of 0.8, and early stopping (50 rounds) on the time holdout. Early stopping selected 141 trees.

Raw `day_of_year` and `days_since_start` are computed and then left out of the model. A split such as `day_of_year <= 304` sends every later date down the same branch, so a tree cannot extrapolate a trend into December. With those features, all 31 December predictions collapsed to one value for that reason. Month, day of week, weekend, and sine/cosine encodings stay, because they repeat inside the range the trees have seen.

4. How the data was split

A random split would train on later dates and score earlier ones. That does not match predicting November and December from history, and it would overstate accuracy.

The split is a single cutoff at 1 September 2025. The model trains on 1 January through 31 August (38,477 rows) and is scored on 1 September through 31 October (9,523 rows, 19.8% of the labeled data). After that check, the final model is refit on all 48,000 labeled rows with the tree count fixed at 142. That refit is what writes `validation_predictions.csv` and the December chart. The holdout is not reused as a second test of the refit model.

5. Holdout results

Metrics are on the original dollar scale, after `expm1`, for September-October 2025.

Model	MAE	RMSE	MAPE	R-squared
Linear regression on distance	\$494.11	\$990.30	23.12%	0.579
XGBoost	\$153.94	\$651.09	6.46%	0.818

XGBoost cuts mean absolute error by 68.8% versus the distance-only line. Distance and haversine distance remain the strongest features. Lane, market index, weight, and the calendar features contribute after them. Residuals across September and October do not show a clear drift. RMSE stays well above MAE because a minority of loads, mostly in the long right tail of `posted_rate`, are still far from the prediction.

6. December 2025 chart

score.py validated both submission files and wrote the chart below from december_predictions.csv. The scenario holds lane, distance, equipment, and weight fixed and changes only the date.

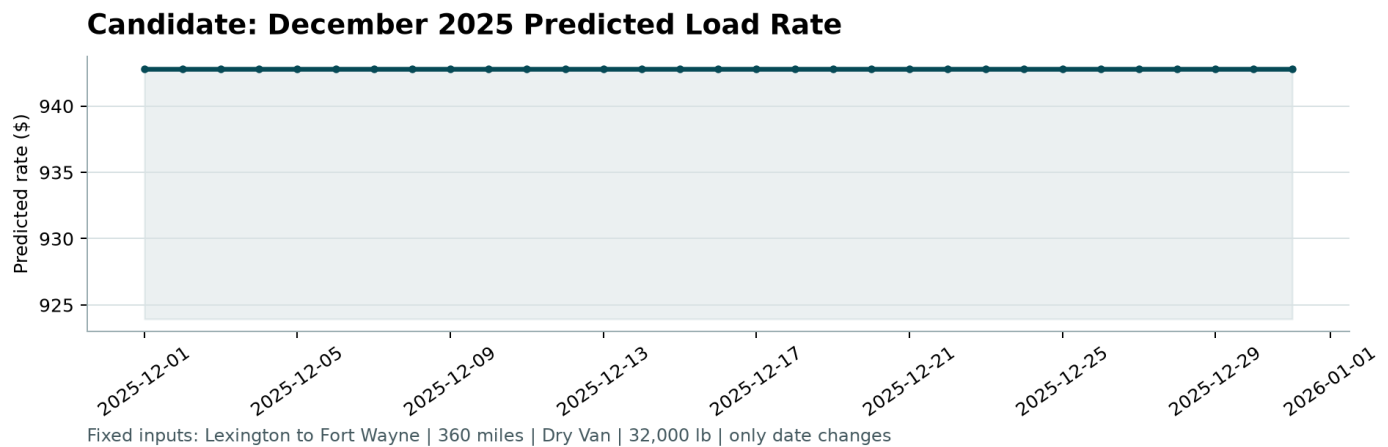


Figure 1. Predicted rate for Lexington to Fort Wayne, December 2025. Produced by score.py.

Every day predicts \$942.81. The December file does not include coordinates, market_index, or quote_signal, so those were reconstructed from the labeled data before scoring:

- Coordinates come from the city lookup. No city had more than one lat/lon pair.
- market_index is a date-level series. A sine/cosine plus linear day-of-year fit on daily means (R-squared 0.70 on observed days) supplies a December value, flagged as estimated.
- quote_signal varies within a day, so a date model is a poor fit. The value used is the historical mean for Lexington, Fort Wayne, and Dry Van (21 past loads on that lane).

Lane, miles, equipment, and weight do not change, and quote_signal is one constant. The small day-to-day move in the estimated market index does not change the leaf this lane falls into, so the curve is flat. That matches the data: once distance and lane are fixed, market conditions move the posted rate very little.

7. Submission files

- validation_predictions.csv: 12,000 rows, IDs aligned to the template, every predicted_rate positive.
- december_predictions.csv: 31 days, original column order, predicted_rate filled in.
- score.py accepted both files and created scorer_results/candidate_december.png.